While the Vercel AI SDK doesn't provide a built-in prompt management system, we can design one that integrates well with any existing application structure.

Figure 5.4 depicts an example architecture of a prompt management system for a generative AI web app that generates different PlantUML diagrams from prompts.

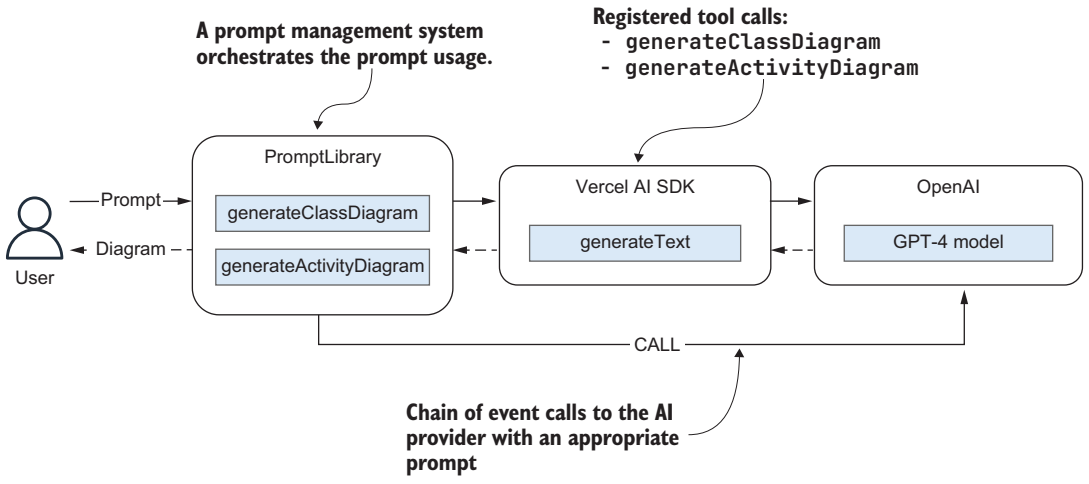



**Figure 5.4 Architecture of the PlantUML diagram-generation system using tool calls in the Vercel AI SDK. The system allows users to request diagrams, and the `PromptLibrary` class handles the generation process by creating prompts, determining the diagram type, and interacting with the Vercel AI SDK to generate the diagram using the AI (OpenAI GPT-4 or other) model and tool calls.**

The basic idea is that the system allows users to request diagrams, and the `PromptLibrary` class handles the generation process. It creates and retrieves prompts, determines the type of diagram requested (class or activity), and calls the appropriate generation method. The `generatePlantUML()` method then interacts with the Vercel AI SDK's `generateText()` function, which sends the prompt to the AI model along with tool calls for generating the specific type of diagram. The AI model processes the request and returns the generated text back through the `generateText()` function, which then returns the generated diagram to the `PromptLibrary` for display to the user.

While not exactly shown in the figure, we can enroll a database for efficiently handling the prompts used in the diagram-generation process. By storing prompts in a dedicated database, the system can dynamically update prompts based on user feedback or performance metrics without the need for redeploying the application. This flexibility allows for continuous improvement of the prompts.

Additionally, registering the `generateClassDiagram` function (which uses a prompt specifically designed for generating class diagrams) and the `generateActivityDiagram` function (which uses a prompt tailored for generating activity diagrams) as tool call